

PostgreSQL CRUD Basics

This handout extends your CRUD foundation by adding **CSV import methods** and **basic SELECT queries** using a real dataset. Before we start, it is imperative that you are familiar with Database Architecture and the ACID Properties. Please read Chapter 4.1 before proceeding to this section.

What is DDL?

DDL (Data Definition Language) refers to SQL commands used to **define and modify database structures**. These covers things like creating tables, altering columns, or dropping objects. Commands such as CREATE, ALTER, and DROP fall under **DDL** because they shape the schema itself.

What is DML?

INSERT, **UPDATE**, and **DELETE** are part of the **DML** (Data Manipulation Language) family commands used to work with the data inside tables, not the structure of the tables themselves. **INSERT** adds new rows into a table; **UPDATE** modifies existing rows based on a condition and **DELETE** removes rows. Unlike **DDL**, which changes the schema, these commands operate on the actual records stored in the database.

As for SELECT, it falls under **DML** just like INSERT, UPDATE, and DELETE. Even though SELECT doesn't modify data, it is still considered DML because it operates on the data stored inside tables. DML is about interacting with data, whether reading or changing it.

Let's Create our First Table

Creating a Table:

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  name varchar(255),  
  age int,  
  email TEXT UNIQUE  
);
```

Table Creation Explanation:

- This table defines users entity with four columns, each serving a specific purpose in the data model.
- The id column uses SERIAL, which automatically generates sequential integer values, making it ideal for a Primary key that uniquely identifies each row.

- The name column uses `varchar(255)`, a flexible text type suitable for names of varying lengths, while age uses an integer type appropriate for numeric attributes.
- The email column is stored as `TEXT` and marked `UNIQUE` thus ensuring no two users can register with the same email address.

Together, these choices illustrate how PostgreSQL data types shape the structure of a table and how constraints enforce data integrity.

For more information, we need to understand what a primary key is. A **primary key** is a column (or set of columns) that uniquely identifies each row in a table thus ensuring that no two records can ever share the same value.

In PostgreSQL, primary keys are often implemented using auto-incrementing types like **SERIAL** or **IDENTITY**, which guarantee that every new row receives a distinct identifier. In data modeling, primary keys are essential because they act as the “anchor points” of your tables. They allow other tables to reference a specific row through **foreign keys**, enable efficient joins, and maintain relational integrity across the entire database.

Without primary keys, relationships between tables would be unreliable, queries would be slower, and enforcing data consistency would be far more difficult.

SERIAL and **IDENTITY** both generate auto-incrementing numbers in PostgreSQL, but they come from two different eras of SQL design.

- **SERIAL** is the older PostgreSQL-specific shortcut: when you declare a column as `SERIAL`, PostgreSQL quietly creates a sequence behind the scenes and sets the column’s default value to pull from that sequence. It works well, but it’s not part of the SQL standard, and the hidden sequence can sometimes surprise beginners when they alter or reset tables.
- **IDENTITY** is the modern, SQL-standard way to define auto-incrementing columns. Instead of creating a separate sequence object manually, PostgreSQL manages it internally and ties it more cleanly to the column.

In practice, `SERIAL` is simpler and widely used in older codebases, while `IDENTITY` is preferred for new designs because it’s standards-compliant, clearer, and behaves more predictably during schema changes or migrations.

PostgreSQL Data Types Overview

Data Type	What It Does	Common Uses
varchar	Stores variable-length text up to a defined limit (e.g., 255 characters).	Names, titles, short labels where you want to enforce a maximum length.
int	Stores whole numbers (positive or negative).	Ages, counts, quantities, numeric attributes without decimals.
text	Stores unlimited-length text with no size restriction.	Emails, descriptions, notes, long strings where length varies widely.
serial	Auto-increments integers using an underlying sequence.	Primary keys, surrogate IDs, unique row identifiers.

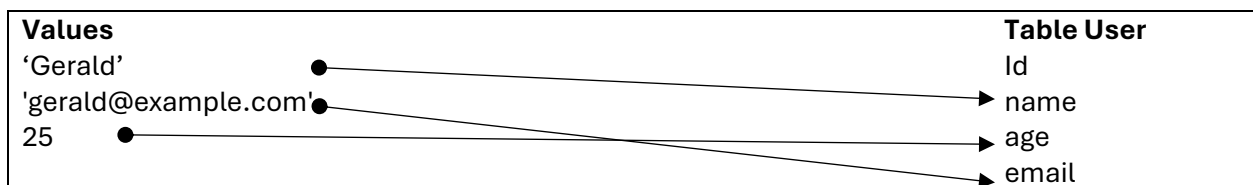
CRUD Operations

CRUD represents the four fundamental operations used to work with data inside a relational database. These operations — **Create**, **Read**, **Update**, and **Delete** — form the core of how applications interact with tables. *Create* adds new records, *Read* retrieves existing data, *Update* modifies stored values, and *Delete* removes records entirely. Together, CRUD defines the full lifecycle of data manipulation and is the foundation of nearly every database-driven system.

CREATE (INSERT)

```
INSERT INTO users (name, email, age)
VALUES ('Gerald', 'gerald@example.com', 25);
```

Explanation: Adds a new row. PostgreSQL enforces the UNIQUE constraint on email.



As you can observe, the values indicated in the query shows its mapping in the corresponding column names. One best practice is to arrange the columns as how they are defined during creation to easily map their corresponding values.

What to do when you encounter an error?

READ (SELECT)

```
SELECT * FROM users;
```

Explanation: Retrieves all rows. Use WHERE to filter.

Expected Output should be:

	id [PK] integer	age integer	name character varying (255)	email text
1	1	25	Gerald	gerald@example.com

UPDATE

```
UPDATE users  
SET name = 'Gerald Anderson'  
WHERE id = 1;
```

Explanation: Updates only rows matching the WHERE condition.

Afterwards, SELECT the users table.

Expected Output should be:

	id [PK] integer	age integer	name character varying (255)	email text
1	1	25	Gerald Anderson	gerald@example.com

DELETE

```
DELETE FROM users  
WHERE id = 1;
```

Explanation: Removes rows. Always include a WHERE clause.

Expected Output should be:

id [PK] integer	age integer	name character varying (255)	email text

Importing Data Using Postgres

Importing a CSV file is the fastest way to **populate a database with real-world data**, especially when starting a new project. In this section, students load the *Food Delivery in Canada (DoorDash)* dataset into PostgreSQL so they have a meaningful table to practice queries on.

Instead of manually inserting hundreds or thousands of rows, CSV import lets PostgreSQL ingest the entire dataset in one step. This gives learners an immediate, realistic dataset to explore, enabling them to practice **SELECT queries**, filtering, aggregation, and data analysis using real delivery fees, ratings, and restaurant information.

Dataset: **Food Delivery in Canada (DoorDash)** Source:

<https://www.kaggle.com/datasets/satoshiss/food-delivery-in-canada-door-dash>

Creating Table Query

1. Delete the first column of the cleaned_full_data.csv
2. Execute the query below:

```
Create table food_delivery(  
    restaurant varchar(255),  
    distance decimal(8,2),  
    star decimal(8,2),  
    num_reviews decimal(8,2),  
    city varchar(255),  
    category_1 varchar(255),  
    category_2 varchar(255)  
);
```

Method 1: Import Using pgAdmin (Task Operations)

1. Right-click your target table.
2. Select **Import/Export**.
3. Choose **Import**.
4. Select your .csv file.
5. Set **Header = Yes**.
6. Set **Delimiter = ,**.
7. Map columns if needed.
8. Click **OK** to import.

Explanation: pgAdmin handles the file parsing and inserts rows automatically.

Method 2: Import Using SQL (COPY Command)

```
COPY food_delivery  
FROM 'C:/path/to/your/file.csv'  
DELIMITER ','  
CSV HEADER;
```

Explanation:

- COPY loads data directly from a CSV file.
- CSV HEADER tells PostgreSQL the first row contains column names.
- File paths must be accessible to the PostgreSQL server.

Basic SELECT Queries

Once the dataset is imported, we will practice these foundational queries.

SELECT All

```
SELECT * FROM food_delivery;
```

Explanation: Returns every row and column.

SELECT With Filter (WHERE)

```
SELECT *  
FROM food_delivery  
WHERE city = 'Toronto';
```

Explanation: Filters rows based on a condition.

SELECT With BETWEEN

```
SELECT *  
FROM food_delivery  
WHERE star BETWEEN 4 AND 5;
```

Explanation: Returns rows within a numeric range.

SELECT With LIKE

```
SELECT *  
FROM food_delivery  
WHERE restaurant LIKE 'Mc%';
```

Explanation: Pattern matching. % means "starts with".

SELECT NOT NULL

```
SELECT *  
FROM food_delivery  
WHERE num_reviews IS NULL;
```

Explanation: Filters out missing values.

Simple Aggregates (SUM, AVG)

```
SELECT  
    SUM(num_reviews) AS total_number_of_views,  
    AVG(num_reviews) AS avg_number_of_views  
FROM food_delivery
```

Explanation: Aggregates summarize numeric columns.

Aggregate with groups

```
SELECT
```

```

category_1,
category_2,
SUM(num_reviews) AS total_number_of_views,
AVG(num_reviews) AS avg_number_of_views
FROM food_delivery
Group by category_1, category_2

```

Explanation: Summarizes data base on clustered data points

CHALLENGE

Write a SELECT query that lists the City and Food Category (use category_2) and the average distance. Filter only stars between 4 and 5 and ignore all reviews and stars that are blank / NULL.

Expected Output:

	city character varying (255) 🔒	food_category character varying (255) 🔒	avg_distance numeric 🔒
1	Edmonton	Tacos	1.10000000000000000000
2	Toronto	Grocery	0.96000000000000000000
3	Toronto	Salads	0.43333333333333333333
4	Montreal	Hot Pot	20.000000000000000000
5	Toronto	Smoothies	0.90000000000000000000
6	Montreal	Snacks	1.15000000000000000000
7	Toronto	Matcha	1.50000000000000000000
8	Vancouver	Heroes	0.50000000000000000000
9	Toronto	Frozen Food	1.40000000000000000000
Total rows: 195		Query complete 00:00:00.227	